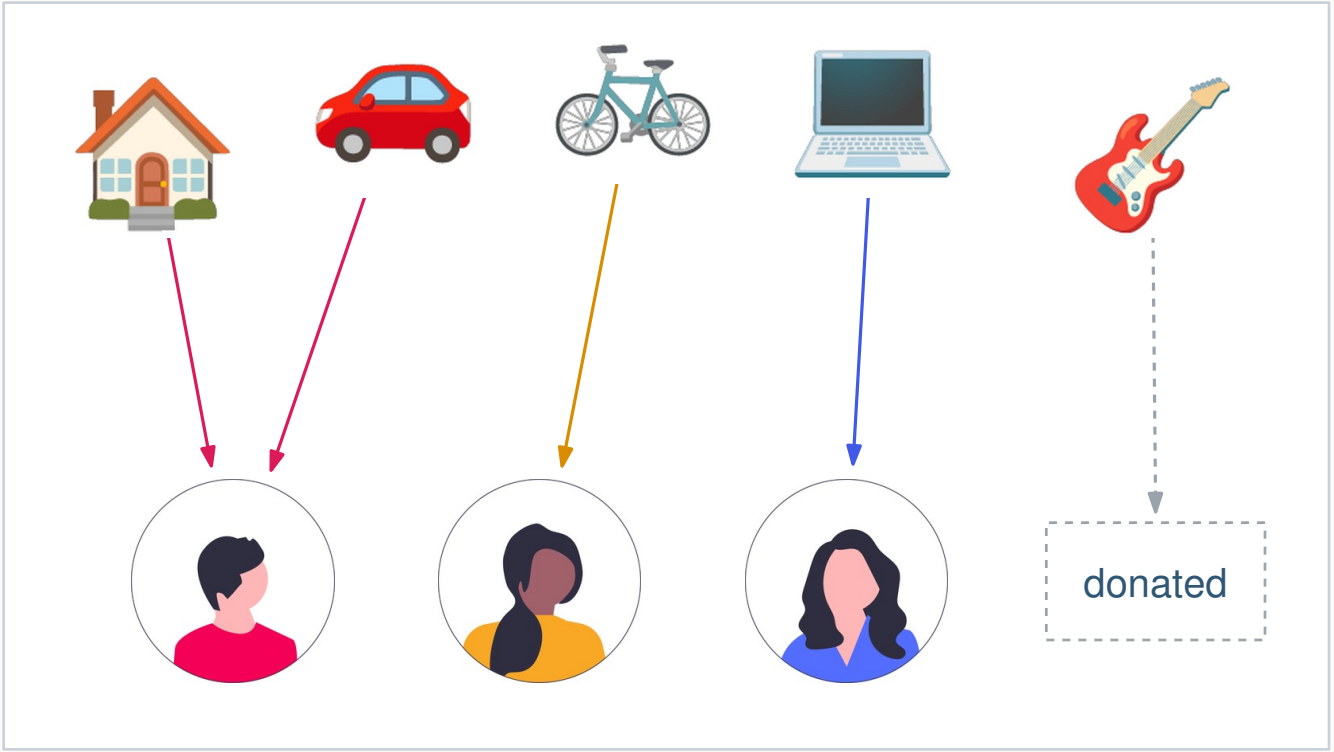
Fair Division

Lecture 04:
Nash Welfare and
Partial EFX



Hannaneh Akrami

UNIVERSITÄT BONN

Fairness and Efficiency

Recall. X is **EF1** if for all agents i, j with $X_j \neq \emptyset$ there is $g \in X_j$ with $v_i(X_i) \geq v_i(X_j \setminus \{g\})$.

EF1 is easy: round robin, envy-cycle elimination, ...

Fairness and Efficiency

Recall. X is **EF1** if for all agents i, j with $X_j \neq \emptyset$ there is $g \in X_j$ with $v_i(X_i) \geq v_i(X_j \setminus \{g\})$.

EF1 is easy: round robin, envy-cycle elimination, ...

Question. Is every EF1 allocation a good allocation?

Fairness and Efficiency

Recall. X is **EF1** if for all agents i, j with $X_j \neq \emptyset$ there is $g \in X_j$ with $v_i(X_i) \geq v_i(X_j \setminus \{g\})$.

EF1 is easy: round robin, envy-cycle elimination, . . .

No! Agent 1 likes only a, agent 2 likes only b.
$X_1 = \{b\}$, $X_2 = \{a\}$ is EF1, but swapping makes **both** happier.

Fairness and Efficiency

Recall. X is **EF1** if for all agents i, j with $X_j \neq \emptyset$ there is $g \in X_j$ with $v_i(X_i) \geq v_i(X_j \setminus \{g\})$.

EF1 is easy: round robin, envy-cycle elimination, . . .

No! Agent 1 likes only a, agent 2 likes only b.
$X_1 = \{b\}$, $X_2 = \{a\}$ is EF1, but swapping makes **both** happier.

Recall. X is **Pareto optimal (PO)** if no allocation Y has $v_i(Y_i) \geq v_i(X_i)$ for all i and $v_j(Y_j) > v_j(X_j)$ for some j.

Fairness and Efficiency

Recall. X is **EF1** if for all agents i, j with $X_j \neq \emptyset$ there is $g \in X_j$ with $v_i(X_i) \geq v_i(X_j \setminus \{g\})$.

EF1 is easy: round robin, envy-cycle elimination, . . .

No! Agent 1 likes only a, agent 2 likes only b.
$X_1 = \{b\}$, $X_2 = \{a\}$ is EF1, but swapping makes **both** happier.

Recall. X is **Pareto optimal (PO)** if no allocation Y has $v_i(Y_i) \geq v_i(X_i)$ for all i and $v_j(Y_j) > v_j(X_j)$ for some j.

Question. Can we have EF1 **and** PO?

Fairness and Efficiency

Recall. X is **EF1** if for all agents i, j with $X_j \neq \emptyset$ there is $g \in X_j$ with $v_i(X_i) \geq v_i(X_j \setminus \{g\})$.

EF1 is easy: round robin, envy-cycle elimination, . . .

No! Agent 1 likes only a, agent 2 likes only b.
$X_1 = \{b\}$, $X_2 = \{a\}$ is EF1, but swapping makes **both** happier.

Recall. X is **Pareto optimal (PO)** if no allocation Y has $v_i(Y_i) \geq v_i(X_i)$ for all i and $v_j(Y_j) > v_j(X_j)$ for some j.

Yes (additive valuations): **maximum Nash welfare**.

Nash Social Welfare

Definition. The **Nash social welfare** of an allocation X is the geometric mean of the values:

$$\mathrm{NW}(X) \;=\; \Big(\prod_{i\in N} v_i(X_i)\Big)^{1/n}$$

An **MNW allocation** maximises NW among all allocations; $\mathrm{opt} = \max_X \mathrm{NW}(X)$.

Nash Social Welfare

Definition. The **Nash social welfare** of an allocation X is the geometric mean of the values:

$$\mathrm{NW}(X) \;=\; \Big(\prod_{i \in N} v_i(X_i)\Big)^{1/n}$$

An **MNW allocation** maximises NW among all allocations; $\mathrm{opt} = \max_X \mathrm{NW}(X)$.

- utilitarian $\sum_i v_i(X_i)$: ignores fairness
- egalitarian $\min_i v_i(X_i)$: ignores efficiency
- Nash $\prod_i v_i(X_i)$: in between

Nash Social Welfare

Definition. The **Nash social welfare** of an allocation X is the geometric mean of the values:

$$\mathrm{NW}(X) \;=\; \Big(\prod_{i\in N} v_i(X_i)\Big)^{1/n}$$

An **MNW allocation** maximises NW among all allocations; $\mathrm{opt} = \max_X \mathrm{NW}(X)$.

- utilitarian $\sum_i v_i(X_i)$: ignores fairness
- egalitarian $\min_i v_i(X_i)$: ignores efficiency
- Nash $\prod_i v_i(X_i)$: in between

Scale invariance: $v_i \to c\,v_i$ multiplies every product by $c \Rightarrow$ same MNW allocations.
Balance: one agent with value $0 \Rightarrow \mathrm{NW} = 0$.

Nash Social Welfare

Definition. The **Nash social welfare** of an allocation X is the geometric mean of the values:

$$\mathrm{NW}(X) \;=\; \Big(\prod_{i\in N} v_i(X_i)\Big)^{1/n}$$

An **MNW allocation** maximises NW among all allocations; $\mathrm{opt} = \max_X \mathrm{NW}(X)$.

- utilitarian $\sum_i v_i(X_i)$: ignores fairness
- egalitarian $\min_i v_i(X_i)$: ignores efficiency
- Nash $\prod_i v_i(X_i)$: in between

Scale invariance: $v_i \to c\,v_i$ multiplies every product by $c \Rightarrow$ same MNW allocations.
Balance: one agent with value $0 \Rightarrow \mathrm{NW} = 0$.

Assumption: some allocation gives every agent positive value, so $\mathrm{opt} > 0$.

Example

		 a	 b	 c
1		6	2	2
2		1	1	1

Example

	 a	 b	 c
1 	6	2	2
2 	1	1	1

Question. Utilitarian optimum?

Example

	 a	 b	 c
1 	6	2	2
2 	1	1	1

Utilitarian: everything to agent 1:
welfare 10, but agent 2 gets 0.

Example

	 a	 b	 c
1 	6	2	2
2 	1	1	1

Utilitarian: everything to agent 1:
welfare 10, but agent 2 gets 0.

Question. MNW allocation?

Example

	 a	 b	 c
1 	6	2	2
2 	1	1	1

Utilitarian: everything to agent 1:
welfare 10, but agent 2 gets 0.

MNW: $X_1 = \{a\}$, $X_2 = \{b, c\}$, product $6 \cdot 2 = 12$.
Other products: $8 \cdot 1$, $10 \cdot 0$, $2 \cdot 2$, $4 \cdot 1$, $\ldots$

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

Question. Why Pareto optimal?

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

PO: a Pareto improvement increases the product.

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

PO: a Pareto improvement increases the product.

EF1: suppose (1) $v_i(X_i) < v_i(X_j \setminus \{g\})$ for all $g \in X_j$.

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

PO: a Pareto improvement increases the product.

EF1: suppose (1) $v_i(X_i) < v_i(X_j \setminus \{g\})$ for all $g \in X_j$.

Question. Which good should move from j to i?

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

PO: a Pareto improvement increases the product.

EF1: suppose (1) $v_i(X_i) < v_i(X_j \setminus \{g\})$ for all $g \in X_j$.

$g^* \in X_j$ **minimising** $\frac{v_j(g)}{v_i(g)}$ over $g \in X_j$ with $v_i(g) > 0$. $\frac{v_j(g^*)}{v_i(g^*)} \leq \frac{v_j(X_j)}{v_i(X_j)}$.

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

PO: a Pareto improvement increases the product.

EF1: suppose (1) $v_i(X_i) < v_i(X_j \setminus \{g\})$ for all $g \in X_j$.

$g^* \in X_j$ **minimising** $\frac{v_j(g)}{v_i(g)}$ over $g \in X_j$ with $v_i(g) > 0$. $\frac{v_j(g^*)}{v_i(g^*)} \leq \frac{v_j(X_j)}{v_i(X_j)}$.

With $a = v_i(g^*)$, $A = v_i(X_i)$, $B = v_i(X_j)$, the product changes by a factor

$$\frac{A+a}{A} \cdot \frac{v_j(X_j)-v_j(g^*)}{v_j(X_j)} \geq \left(1+\frac{a}{A}\right)\left(1-\frac{a}{B}\right).$$

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

PO: a Pareto improvement increases the product.

EF1: suppose (1) $v_i(X_i) < v_i(X_j \setminus \{g\})$ for all $g \in X_j$.

$g^* \in X_j$ **minimising** $\frac{v_j(g)}{v_i(g)}$ over $g \in X_j$ with $v_i(g) > 0$. $\frac{v_j(g^*)}{v_i(g^*)} \leq \frac{v_j(X_j)}{v_i(X_j)}$.

With $a = v_i(g^*)$, $A = v_i(X_i)$, $B = v_i(X_j)$, the product changes by a factor

$$\frac{A+a}{A} \cdot \frac{v_j(X_j)-v_j(g^*)}{v_j(X_j)} \geq \left(1+\frac{a}{A}\right)\left(1-\frac{a}{B}\right).$$

Question. Why is this larger than 1?

MNW Is EF1 and Pareto Optimal

Theorem (Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016).
For additive valuations, every MNW allocation is EF1 and Pareto optimal.

Let X be MNW; $v_i(X_i) > 0$ for all i.

PO: a Pareto improvement increases the product.

EF1: suppose (1) $v_i(X_i) < v_i(X_j \setminus \{g\})$ for all $g \in X_j$.

$g^* \in X_j$ **minimising** $\frac{v_j(g)}{v_i(g)}$ over $g \in X_j$ with $v_i(g) > 0$. $\frac{v_j(g^*)}{v_i(g^*)} \leq \frac{v_j(X_j)}{v_i(X_j)}$.

With $a = v_i(g^*)$, $A = v_i(X_i)$, $B = v_i(X_j)$, the product changes by a factor

$$\frac{A+a}{A} \cdot \frac{v_j(X_j) - v_j(g^*)}{v_j(X_j)} \geq \left(1 + \frac{a}{A}\right)\left(1 - \frac{a}{B}\right).$$

(1) with g^*: $A < B - a$, so $\left(1 + \frac{a}{A}\right)\left(1 - \frac{a}{B}\right) > \left(1 + \frac{a}{B-a}\right)\left(1 - \frac{a}{B}\right) = 1$.

Contradiction to the maximality of X.

MNW Need Not Be EFX

Recall (EFX, introduced in the same paper). $v_i(X_i) \geq v_i(X_j \setminus \{g\})$ for all i, j and **all** $g \in X_j$.

	 g	 p_1	 p_2
1 	1	$1-\varepsilon$	$\frac{\varepsilon}{4}$
2 	1	$\frac{\varepsilon}{4}$	$1-\varepsilon$

MNW Need Not Be EFX

Recall (EFX, introduced in the same paper). $v_i(X_i) \geq v_i(X_j \setminus \{g\})$ for all i, j and **all** $g \in X_j$.

	 g	 p_1	 p_2
1 	1	$1-\varepsilon$	$\frac{\varepsilon}{4}$
2 	1	$\frac{\varepsilon}{4}$	$1-\varepsilon$

Question. MNW allocation?

MNW Need Not Be EFX

Recall (EFX, introduced in the same paper). $v_i(X_i) \geq v_i(X_j \setminus \{g\})$ for all i, j and **all** $g \in X_j$.

	 g	 p_1	 p_2
1 	1	$1-\varepsilon$	$\frac{\varepsilon}{4}$
2 	1	$\frac{\varepsilon}{4}$	$1-\varepsilon$

$X_1 = \{g, p_1\}$, $X_2 = \{p_2\}$
$\mathrm{NW}^2 = (2-\varepsilon)(1-\varepsilon)$ (or symmetric)

MNW Need Not Be EFX

Recall (EFX, introduced in the same paper). $v_i(X_i) \geq v_i(X_j \setminus \{g\})$ for all i, j and **all** $g \in X_j$.

	 g	 p_1	 p_2
1 	1	$1-\varepsilon$	$\frac{\varepsilon}{4}$
2 	1	$\frac{\varepsilon}{4}$	$1-\varepsilon$

$X_1 = \{g, p_1\}$, $X_2 = \{p_2\}$
$\mathrm{NW}^2 = (2-\varepsilon)(1-\varepsilon)$ (or symmetric)

Question. Is it EFX?

MNW Need Not Be EFX

Recall (EFX, introduced in the same paper). $v_i(X_i) \geq v_i(X_j \setminus \{g\})$ for all i, j and **all** $g \in X_j$.

	 g	 p_1	 p_2
1 	1	$1-\varepsilon$	$\frac{\varepsilon}{4}$
2 	1	$\frac{\varepsilon}{4}$	$1-\varepsilon$

$X_1 = \{g, p_1\}$, $X_2 = \{p_2\}$
$\mathrm{NW}^2 = (2-\varepsilon)(1-\varepsilon)$ (or symmetric)

No: $v_2(X_1 \setminus \{p_1\}) = 1 > 1-\varepsilon = v_2(X_2)$.

MNW Need Not Be EFX

Recall (EFX, introduced in the same paper). $v_i(X_i) \geq v_i(X_j \setminus \{g\})$ for all i, j and **all** $g \in X_j$.

	 g	 p_1	 p_2
1 	1	$1-\varepsilon$	$\frac{\varepsilon}{4}$
2 	1	$\frac{\varepsilon}{4}$	$1-\varepsilon$

$X_1 = \{g, p_1\}$, $X_2 = \{p_2\}$
$\mathrm{NW}^2 = (2-\varepsilon)(1-\varepsilon)$ (or symmetric)

No: $v_2(X_1 \setminus \{p_1\}) = 1 > 1-\varepsilon = v_2(X_2)$.

Way out: **donate** some goods!

Partial EFX with High Nash Welfare

A **partial allocation** Y allocates a subset of the goods; the rest is **donated**.

Partial EFX with High Nash Welfare

A **partial allocation** Y allocates a subset of the goods; the rest is **donated**.

Theorem (Caragiannis, Gravin, Huang 2019).
Let X be an MNW allocation (additive valuations). There is an EFX partial allocation Y with $Y_i \subseteq X_i$ for all i and

$$\mathrm{NW}(Y) \;\geq\; 2^{-(1-1/n)} \cdot \mathrm{opt} \;>\; \tfrac{1}{2}\,\mathrm{opt}.$$

Given X, such a Y can be computed in polynomial time.

Partial EFX with High Nash Welfare

A **partial allocation** Y allocates a subset of the goods; the rest is **donated**.

Theorem (Caragiannis, Gravin, Huang 2019).
Let X be an MNW allocation (additive valuations). There is an EFX partial allocation Y with $Y_i \subseteq X_i$ for all i and

$$\mathrm{NW}(Y) \;\geq\; 2^{-(1-1/n)} \cdot \mathrm{opt} \;>\; \tfrac{1}{2}\,\mathrm{opt}.$$

Given X, such a Y can be computed in polynomial time.

Stronger: $v_i(Y_i) \geq \frac{1}{2} v_i(X_i)$ for all i, and $Y_{i_0} = X_{i_0}$ for some i_0.

The Feasibility Graph

Bundles $Z_j \subseteq X_j$, initially $Z_j = X_j$. Z_j is **touched** once a good was removed from it.

The Feasibility Graph

Bundles $Z_j \subseteq X_j$, initially $Z_j = X_j$. Z_j is **touched** once a good was removed from it.

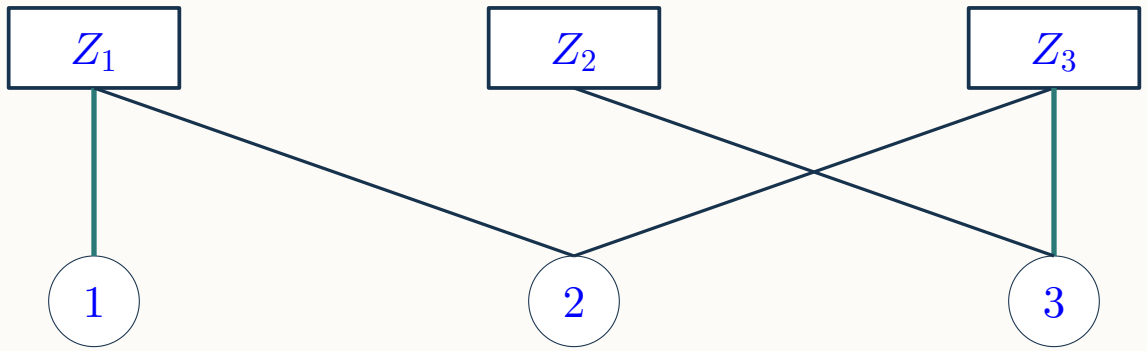
Z_j is **EFX-feasible** for i if $v_i(Z_j) \geq v_i(Z_k \setminus \{g\})$ for all k and $g \in Z_k$.

The Feasibility Graph

Bundles $Z_j \subseteq X_j$, initially $Z_j = X_j$. Z_j is **touched** once a good was removed from it.

Z_j is **EFX-feasible** for i if $v_i(Z_j) \geq v_i(Z_k \setminus \{g\})$ for all k and $g \in Z_k$.

Feasibility graph G: agents vs. bundles, edge (i, Z_j) if Z_j is EFX-feasible for i and, if $j \neq i$, $v_i(Z_j) > v_i(Z_i)$. Edges (i, Z_i): **identity edges**.



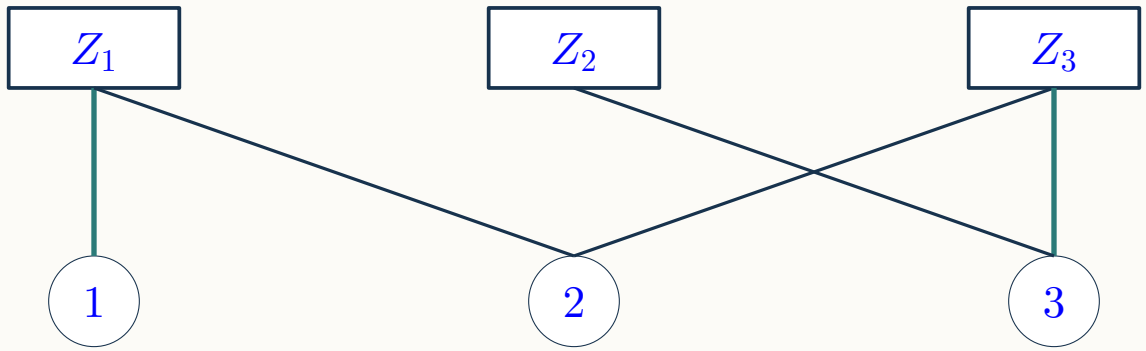
identity edges

The Feasibility Graph

Bundles $Z_j \subseteq X_j$, initially $Z_j = X_j$. Z_j is **touched** once a good was removed from it.

Z_j is **EFX-feasible** for i if $v_i(Z_j) \geq v_i(Z_k \setminus \{g\})$ for all k and $g \in Z_k$.

Feasibility graph G: agents vs. bundles, edge (i, Z_j) if Z_j is EFX-feasible for i and, if $j \neq i$, $v_i(Z_j) > v_i(Z_i)$. Edges (i, Z_i): **identity edges**.



identity edges

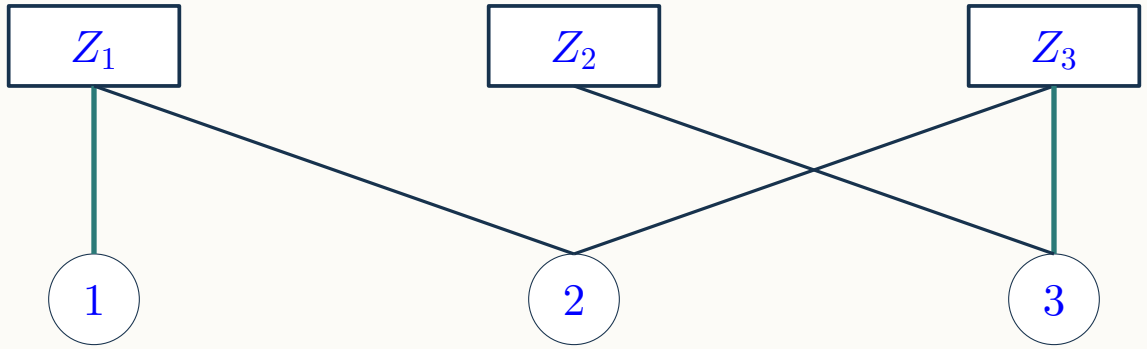
Question. What does a perfect matching give?

The Feasibility Graph

Bundles $Z_j \subseteq X_j$, initially $Z_j = X_j$. Z_j is **touched** once a good was removed from it.

Z_j is **EFX-feasible** for i if $v_i(Z_j) \geq v_i(Z_k \setminus \{g\})$ for all k and $g \in Z_k$.

Feasibility graph G: agents vs. bundles, edge (i, Z_j) if Z_j is EFX-feasible for i and, if $j \neq i$, $v_i(Z_j) > v_i(Z_i)$. Edges (i, Z_i): **identity edges**.



identity edges

Perfect matching $M \Rightarrow Y_i = M(i)$ is **EFX**: every agent holds an EFX-feasible bundle.

The Algorithm

Algorithm (Caragiannis, Gravin, Huang 2019). Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) subject to (a), maximum number of identity edges;
   (c) subject to (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

The Algorithm

Algorithm (Caragiannis, Gravin, Huang 2019). Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) subject to (a), maximum number of identity edges;
   (c) subject to (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

An unmatched agent points at the bundle she likes most **after removing its least valuable good**.

The Algorithm

Algorithm (Caragiannis, Gravin, Huang 2019). Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) subject to (a), maximum number of identity edges;
   (c) subject to (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

An unmatched agent points at the bundle she likes most **after removing its least valuable good**.

Each iteration takes polynomial time and there are at most $m + 1$ iterations (Exercise 3).

A Covering Matching Exists

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Algorithm. Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) s.t. (a), maximum number of identity edges;
   (c) s.t. (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

M covers the touched bundles T; i^* unmatched;
$Z'_{j^*} = Z_{j^*} \setminus \{g^*\}$; G': new graph.

A Covering Matching Exists

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Algorithm. Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) s.t. (a), maximum number of identity edges;
   (c) s.t. (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

M covers the touched bundles T; i^* unmatched; $Z'_{j^*} = Z_{j^*} \setminus \{g^*\}$; G': new graph.

(1) $(i^*, Z_{i^*}) \notin G$: else add it, or swap it for (k, Z_{i^*}): more identity edges, contradicting (b).

A Covering Matching Exists

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Algorithm. Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) s.t. (a), maximum number of identity edges;
   (c) s.t. (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

M covers the touched bundles T; i^* unmatched; $Z'_{j^*} = Z_{j^*} \setminus \{g^*\}$; G': new graph.

(1) $(i^*, Z_{i^*}) \notin G$: else add it, or swap it for (k, Z_{i^*}): more identity edges, contradicting (b).

(2) $(i^*, Z'_{j^*}) \in G'$: by (1), Z_{i^*} is not EFX-feasible for i^*, so $v_{i^*}(Z'_{j^*}) = \max_{k,\, g \in Z_k} v_{i^*}(Z_k \setminus \{g\}) > v_{i^*}(Z_{i^*})$; hence $j^* \neq i^*$.

A Covering Matching Exists

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Algorithm. Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) s.t. (a), maximum number of identity edges;
   (c) s.t. (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

M covers the touched bundles T; i^* unmatched; $Z'_{j^*} = Z_{j^*} \setminus \{g^*\}$; G': new graph.

(1) $(i^*, Z_{i^*}) \notin G$: else add it, or swap it for (k, Z_{i^*}): more identity edges, contradicting (b).

(2) $(i^*, Z'_{j^*}) \in G'$: by (1), Z_{i^*} is not EFX-feasible for i^*, so $v_{i^*}(Z'_{j^*}) = \max_{k,\, g \in Z_k} v_{i^*}(Z_k \setminus \{g\}) > v_{i^*}(Z_{i^*})$; hence $j^* \neq i^*$.

(3) All other edges (i, Z_j), $j \neq j^*$, survive in G': bundles only shrink.

A Covering Matching Exists

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Algorithm. Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) s.t. (a), maximum number of identity edges;
   (c) s.t. (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

M covers the touched bundles T; i^* unmatched; $Z'_{j^*} = Z_{j^*} \setminus \{g^*\}$; G': new graph.

(1) $(i^*, Z_{i^*}) \notin G$: else add it, or swap it for (k, Z_{i^*}): more identity edges, contradicting (b).

(2) $(i^*, Z'_{j^*}) \in G'$: by (1), Z_{i^*} is not EFX-feasible for i^*, so $v_{i^*}(Z'_{j^*}) = \max_{k,\, g \in Z_k} v_{i^*}(Z_k \setminus \{g\}) > v_{i^*}(Z_{i^*})$; hence $j^* \neq i^*$.

(3) All other edges (i, Z_j), $j \neq j^*$, survive in G': bundles only shrink.

(4) Z_{j^*} is matched in M, say to k: otherwise add (i^*, Z_{j^*}) (an edge of G, as in (2)), contradicting (c).

A Covering Matching Exists

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Algorithm. Start with $Z_j = X_j$. Repeat:

1. compute a matching M in G with priorities:
   (a) every touched bundle is matched;
   (b) s.t. (a), maximum number of identity edges;
   (c) s.t. (a) and (b), maximum size;
2. if M is perfect, return $Y_i = M(i)$;
3. else, for an unmatched agent i^*: take the bundle Z_{j^*} maximising $v_{i^*}(Z) - \min_{g \in Z} v_{i^*}(g)$, and **donate** its least valuable good (for i^*).

M covers the touched bundles T; i^* unmatched; $Z'_{j^*} = Z_{j^*} \setminus \{g^*\}$; G': new graph.

(1) $(i^*, Z_{i^*}) \notin G$: else add it, or swap it for (k, Z_{i^*}): more identity edges, contradicting (b).

(2) $(i^*, Z'_{j^*}) \in G'$: by (1), Z_{i^*} is not EFX-feasible for i^*, so $v_{i^*}(Z'_{j^*}) = \max_{k,\, g \in Z_k} v_{i^*}(Z_k \setminus \{g\}) > v_{i^*}(Z_{i^*})$; hence $j^* \neq i^*$.

(3) All other edges (i, Z_j), $j \neq j^*$, survive in G': bundles only shrink.

(4) Z_{j^*} is matched in M, say to k: otherwise add (i^*, Z_{j^*}) (an edge of G, as in (2)), contradicting (c).

$M' = \big(M \setminus \{(k, Z_{j^*})\}\big) \cup \{(i^*, Z'_{j^*})\}$ covers $T \cup \{Z'_{j^*}\}$.

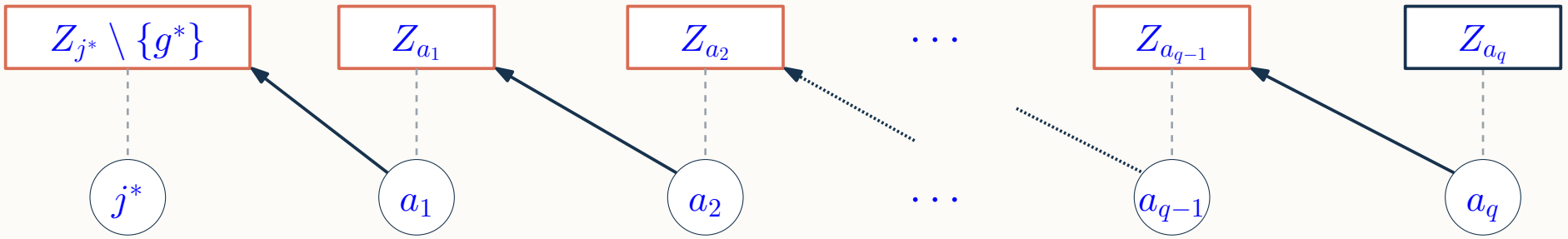
Every Agent Keeps Half

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

Every Agent Keeps Half

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

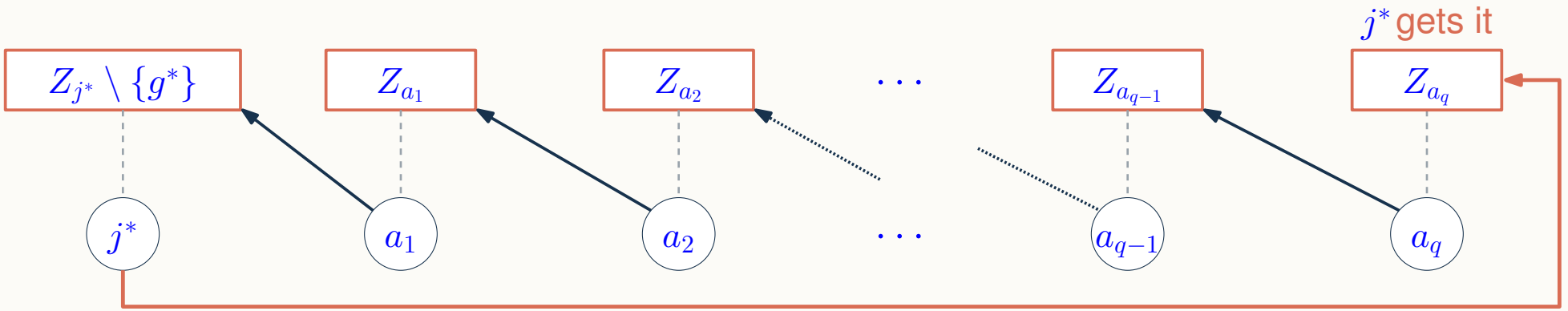
First violation: g^* is removed from Z_{j^*} and $v_{j^*}(Z_{j^*} \setminus \{g^*\}) < \frac{1}{2} v_{j^*}(X_{j^*})$. M' (Lemma 1) covers touched bundles, $M'(Z_{j^*} \setminus \{g^*\}) = i^* \neq j^*$.
Walk backwards: $a_1 = M'(Z_{j^*} \setminus \{g^*\})$, $a_{r+1} = M'(Z_{a_r})$ while Z_{a_r} is touched.



Every Agent Keeps Half

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

First violation: g^* is removed from Z_{j^*} and $v_{j^*}(Z_{j^*} \setminus \{g^*\}) < \frac{1}{2} v_{j^*}(X_{j^*})$. M' (Lemma 1) covers touched bundles, $M'(Z_{j^*} \setminus \{g^*\}) = i^* \neq j^*$.
Walk backwards: $a_1 = M'(Z_{j^*} \setminus \{g^*\})$, $a_{r+1} = M'(Z_{a_r})$ while Z_{a_r} is touched.

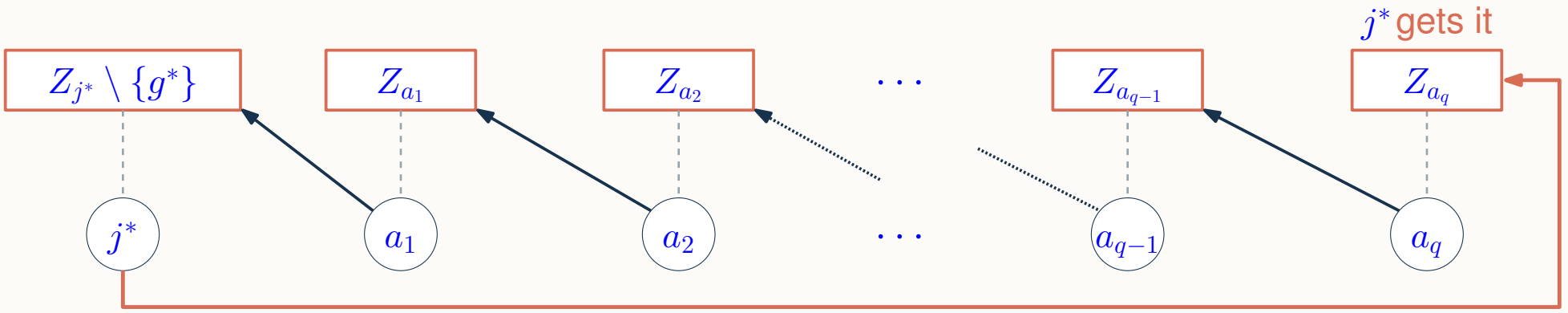


Question. **Cycle** (the walk returns to j^*): contradiction?

Every Agent Keeps Half

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

First violation: g^* is removed from Z_{j^*} and $v_{j^*}(Z_{j^*} \setminus \{g^*\}) < \frac{1}{2} v_{j^*}(X_{j^*})$. M' (Lemma 1) covers touched bundles, $M'(Z_{j^*} \setminus \{g^*\}) = i^* \neq j^*$.
Walk backwards: $a_1 = M'(Z_{j^*} \setminus \{g^*\})$, $a_{r+1} = M'(Z_{a_r})$ while Z_{a_r} is touched.

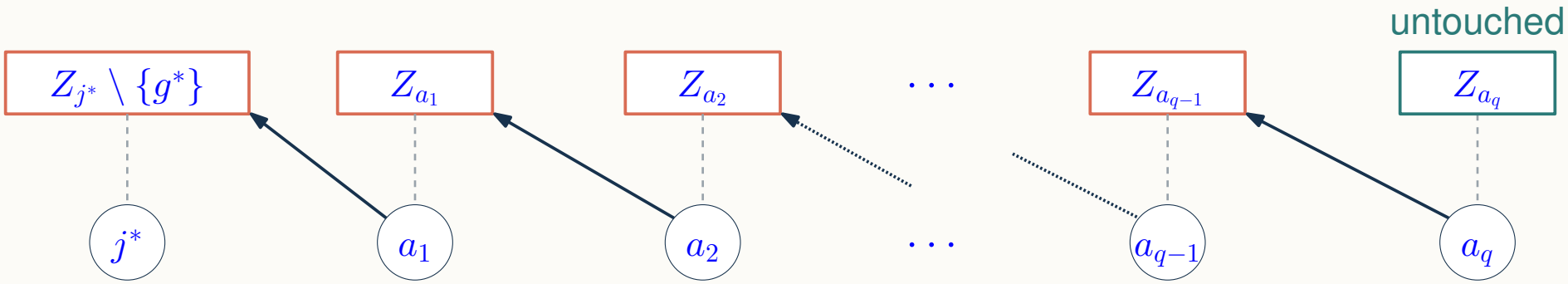


Cycle: each a swaps Z_a for $M'(a)$: $\widehat{X}_a = (X_a \setminus Z_a) \cup M'(a)$; all strictly gain $\Rightarrow$ NW increases.

Every Agent Keeps Half

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

First violation: g^* is removed from Z_{j^*} and $v_{j^*}(Z_{j^*} \setminus \{g^*\}) < \frac{1}{2} v_{j^*}(X_{j^*})$. M' (Lemma 1) covers touched bundles, $M'(Z_{j^*} \setminus \{g^*\}) = i^* \neq j^*$.
Walk backwards: $a_1 = M'(Z_{j^*} \setminus \{g^*\})$, $a_{r+1} = M'(Z_{a_r})$ while Z_{a_r} is touched.

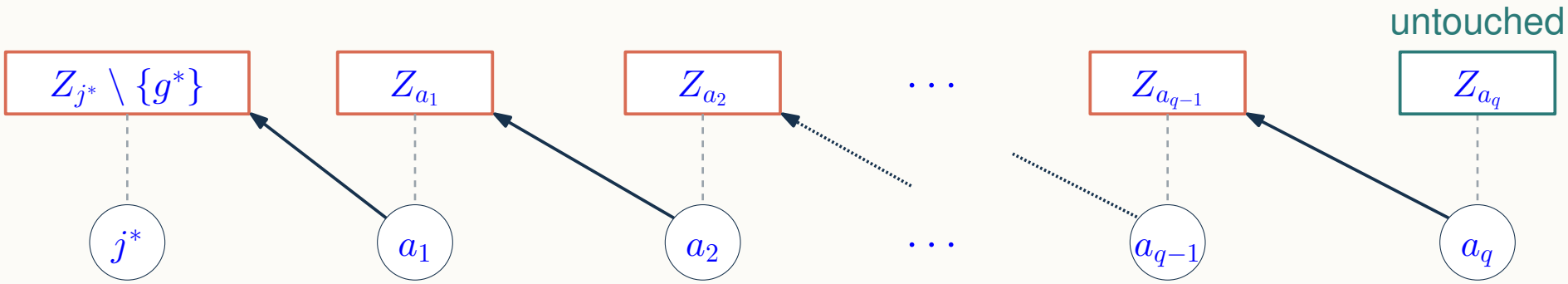


Cycle: each a swaps Z_a for $M'(a)$: $\widehat{X}_a = (X_a \setminus Z_a) \cup M'(a)$; all strictly gain $\Rightarrow$ NW increases.
Question. **Path** (ends at untouched $Z_{a_q} = X_{a_q}$): contradiction?

Every Agent Keeps Half

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

First violation: g^* is removed from Z_{j^*} and $v_{j^*}(Z_{j^*} \setminus \{g^*\}) < \frac{1}{2} v_{j^*}(X_{j^*})$. M' (Lemma 1) covers touched bundles, $M'(Z_{j^*} \setminus \{g^*\}) = i^* \neq j^*$.
Walk backwards: $a_1 = M'(Z_{j^*} \setminus \{g^*\})$, $a_{r+1} = M'(Z_{a_r})$ while Z_{a_r} is touched.



Cycle: each a swaps Z_a for $M'(a)$: $\widehat{X}_a = (X_a \setminus Z_a) \cup M'(a)$; all strictly gain $\Rightarrow$ NW increases.

Path: $\widehat{X}_{a_q} = X_{a_q} \cup M'(a_q)$, $\widehat{X}_{a_r} = (X_{a_r} \setminus Z_{a_r}) \cup M'(a_r)$, $\widehat{X}_{j^*} = X_{j^*} \setminus (Z_{j^*} \setminus \{g^*\})$.
Factors: > 2 for a_q, > 1 for a_r, $> \frac{1}{2}$ for $j^* \Rightarrow$ NW increases: **contradiction**.

One Bundle Stays Untouched

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

One Bundle Stays Untouched

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

Suppose all are touched; look at the iteration touching the last one, Z_{j^*}. Then M covers the $n-1$ touched bundles (Lemma 1), and Z_{j^*} is matched (else adding (i^*, Z_{j^*}) gives a larger matching) $\Rightarrow$ M is **perfect** $\Rightarrow$ we would have stopped.

Proof of the Theorem

Theorem (Caragiannis, Gravin, Huang 2019).
For an MNW allocation X, the algorithm returns an EFX partial allocation Y with $Y_i \subseteq X_i$ and $\mathrm{NW}(Y) \geq 2^{-(1-1/n)} \operatorname{opt}$.

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

Proof of the Theorem

Theorem (Caragiannis, Gravin, Huang 2019).
For an MNW allocation X, the algorithm returns an EFX partial allocation Y with $Y_i \subseteq X_i$ and $\mathrm{NW}(Y) \geq 2^{-(1-1/n)} \operatorname{opt}$.

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

Question. Why is Y EFX?

Proof of the Theorem

Theorem (Caragiannis, Gravin, Huang 2019).
For an MNW allocation X, the algorithm returns an EFX partial allocation Y with $Y_i \subseteq X_i$ and $\mathrm{NW}(Y) \geq 2^{-(1-1/n)} \operatorname{opt}$.

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

EFX: it stops with a perfect matching of G, so every agent gets an EFX-feasible bundle (Lemma 1: the algorithm never gets stuck).

Proof of the Theorem

Theorem (Caragiannis, Gravin, Huang 2019).
For an MNW allocation X, the algorithm returns an EFX partial allocation Y with $Y_i \subseteq X_i$ and $\mathrm{NW}(Y) \geq 2^{-(1-1/n)} \operatorname{opt}$.

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

EFX: it stops with a perfect matching of G, so every agent gets an EFX-feasible bundle (Lemma 1: the algorithm never gets stuck).

Question. Why is $Y_i \subseteq X_i$?

Proof of the Theorem

Theorem (Caragiannis, Gravin, Huang 2019).
For an MNW allocation X, the algorithm returns an EFX partial allocation Y with $Y_i \subseteq X_i$ and $\mathrm{NW}(Y) \geq 2^{-(1-1/n)} \operatorname{opt}$.

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

EFX: it stops with a perfect matching of G, so every agent gets an EFX-feasible bundle (Lemma 1: the algorithm never gets stuck).

$Y_i = Z_i$**:** otherwise the perfect matching contains a cycle of non-identity edges $\Rightarrow$ exchange increases $\mathrm{NW}(X)$.

Proof of the Theorem

Theorem (Caragiannis, Gravin, Huang 2019).
For an MNW allocation X, the algorithm returns an EFX partial allocation Y with $Y_i \subseteq X_i$ and $\mathrm{NW}(Y) \geq 2^{-(1-1/n)} \operatorname{opt}$.

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

EFX: it stops with a perfect matching of G, so every agent gets an EFX-feasible bundle (Lemma 1: the algorithm never gets stuck).

$Y_i = Z_i$**:** otherwise the perfect matching contains a cycle of non-identity edges $\Rightarrow$ exchange increases $\mathrm{NW}(X)$.

Question. Nash welfare?

Proof of the Theorem

Theorem (Caragiannis, Gravin, Huang 2019).
For an MNW allocation X, the algorithm returns an EFX partial allocation Y with $Y_i \subseteq X_i$ and $\mathrm{NW}(Y) \geq 2^{-(1-1/n)} \operatorname{opt}$.

Lemma 1.
In every iteration, G has a matching that covers all touched bundles.

Lemma 2.
Throughout the algorithm, $v_i(Z_i) \geq \frac{1}{2} v_i(X_i)$ for every agent i.

Lemma 3.
When the algorithm stops, at least one bundle is untouched.

EFX: it stops with a perfect matching of G, so every agent gets an EFX-feasible bundle (Lemma 1: the algorithm never gets stuck).

$Y_i = Z_i$**:** otherwise the perfect matching contains a cycle of non-identity edges $\Rightarrow$ exchange increases $\mathrm{NW}(X)$.

NW: $v_i(Y_i) \geq \frac{1}{2} v_i(X_i)$ for all i (Lemma 2) and $Y_{i_0} = X_{i_0}$ for some i_0 (Lemma 3). So

$$\mathrm{NW}(Y)^n \;\geq\; 2^{-(n-1)} \operatorname{opt}^n.$$

Remarks

- Computing an MNW allocation is **NP-hard** (Exercise 2). The algorithm can be modified to start from a ρ-approximate MNW allocation instead: it then computes, in polynomial time, an EFX partial allocation whose Nash welfare is within a factor 2ρ of opt.

Remarks

- Computing an MNW allocation is **NP-hard** (Exercise 2). The algorithm can be modified to start from a ρ-approximate MNW allocation instead: it then computes, in polynomial time, an EFX partial allocation whose Nash welfare is within a factor 2ρ of opt.

- Here many goods may be donated, and agents may envy the donated goods.

Chaudhury, Kavitha, Mehlhorn, Sgouritsa 2020: fewer than n donated goods, and nobody envies them (monotone valuations).

Summary

Allocation	Fairness	Efficiency
MNW (complete)	EF1	PO, optimal NW
Caragiannis, Gravin, Huang 2019 (partial)	EFX	$\mathrm{NW} \geq 2^{-(1-1/n)} \operatorname{opt}$

Exercises

Q 1 (The factor is tight). Show that the factor $2^{-(1-1/n)}$ cannot be improved: for every n and $\delta > 0$ there is an instance with additive valuations in which every EFX partial allocation Y satisfies $\mathrm{NW}(Y) < (2^{-(1-1/n)} + \delta)\,\mathrm{opt}$.

Q 2 (Computing MNW is hard).
(a) Show that deciding whether an instance has an allocation with $\mathrm{NW} \geq k$ is NP-hard, even for $n = 2$ agents with the **same** additive valuation.
(b) Show that it is strongly NP-hard for n agents with the same additive valuation.

Q 3 (Running time). Show that, given X, the algorithm of Caragiannis, Gravin, Huang runs in polynomial time:
(a) the feasibility graph G can be computed in polynomial time;
(b) a matching as in step 1, with priorities (a), (b), (c), can be computed in polynomial time;
(c) the algorithm stops after at most $m + 1$ iterations.

Hints.
Q 2: (a) Reduce from Partition: given positive integers $a_1, \ldots, a_m$, is there $T \subseteq [m]$ with $\sum_{j \in T} a_j = \sum_{j \notin T} a_j$?
(b) Reduce from 3-Partition (strongly NP-hard): given positive integers $a_1, \ldots, a_{3n}$ with $\sum_j a_j = nB$ and $B/4 < a_j < B/2$, can they be split into n groups, each of sum B?